	Carátula para entrega de prácticas	
Facultad de Ingeniería		Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): José Rodrigo Moreno Lopez

Asignatura: EDA II

Grupo: 11

No de Práctica(s): Práctica #8

Integrante(s): 120002850

No. de lista o brigada:

Semestre: 2027-1

Fecha de entrega: 05 de noviembre de 2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **4**

1.1. Planteamiento del problema 4

1.2. Motivación 4

1.3. Objetivos 4

2. Marco Teórico **5**

3. Desarrollo **6**

3.1. Investigación y análisis de la consulta 6

3.2. Interfaz gráfica de login 7

3.3. Contrato de huella de carbono 8

3.4. Clase Edificio 9

3.5. Clase Auto 10

3.6. Clase Bicicleta 11

3.7. Clase principal y despacho polimórfico 12

4. Resultados **13**

4.1. Verificación cuantitativa de emisiones 13

4.2. Ejecución desde classpath y archivo JAR 14

4.3. Documentación técnica y estructura de entrega 15

5. Conclusiones **16**

1. Introducción

1.1. Planteamiento del problema

La práctica solicita resolver dos necesidades fundamentales relacionadas con el paradigma orientado a objetos en Java. La primera consiste en plantear una interfaz gráfica interactiva para capturar un nombre de usuario y una contraseña mediante componentes de Swing. La segunda requiere modelar tres entidades diferenciadas —un edificio, un automóvil y una bicicleta— a través de clases que implementen una misma interfaz denominada `HuellaCarbono`. Cada objeto debe estimar su huella a partir de variables propias de su dominio operativo, mientras que la clase principal debe organizar las instancias en una colección de referencias comunes para evidenciar el despacho polimórfico en tiempo de ejecución.

1.2. Motivación

El ejercicio permite contrastar el desacoplamiento que brindan las interfaces frente a la herencia tradicional. Una interfaz define un contrato de operaciones sin imponer estado compartido ni restringir la mecánica de cálculo a una fórmula única. Así, el edificio vincula consumo eléctrico y factor de emisión; el automóvil procesa distancia recorrida, rendimiento de combustible y factor de gasolina; y la bicicleta expresa que su operación no produce emisiones directas en el alcance didáctico delimitado. Por otra parte, la implementación de la interfaz gráfica complementa el aprendizaje al demostrar la manipulación segura de contraseñas y el despacho de eventos mediante oyentes.

1.3. Objetivos

- Diseñar e implementar el contrato de la interfaz `HuellaCarbono` junto con sus clases concretas (`Edificio`, `Auto` y `Bicicleta`) para calcular emisiones según sus propios atributos y fórmulas de dominio.
- Demostrar el polimorfismo dinámico en Java mediante el almacenamiento de instancias en una colección homogénea `ArrayList<HuellaCarbono>` y su invocación uniforme durante el recorrido.
- Construir una interfaz gráfica interactiva de inicio de sesión con componentes Swing (`LoginGUI`) y verificar exhaustivamente la solución mediante compilación limpia, ejecución reproducible, generación de Javadoc y empaquetado en archivo JAR.

2. Marco Teórico

Una interfaz en Java constituye un contrato abstracto de operaciones que las clases concretas se comprometen a satisfacer. La especificación oficial del lenguaje establece que una clase puede implementar múltiples interfaces y que cualquier referencia tipada a la interfaz puede emplearse para invocar sus métodos declarados [1]. En esta práctica, `HuellaCarbono` define únicamente la firma `double getHuellaCarbono()`, delegando enteramente en cada clase la lógica de evaluación. La resolución del método efectivo tiene lugar de manera dinámica durante la ejecución, materializando el principio de ligadura tardía y polimorfismo.

El principio de encapsulamiento garantiza que el estado de cada entidad permanezca resguardado dentro de la clase correspondiente. De este modo, variables tales como consumo anual, kilometraje, rendimiento y factores de emisión se declaran como atributos privados, proveyendo métodos accesores públicos y la operación de cálculo exigida por el contrato. Se descarta deliberadamente el uso de herencia simple porque un edificio, un automóvil y una bicicleta no comparten un antepasado semántico común en el mundo real; comparten exclusivamente una capacidad operativa de medición ambiental.

En el ámbito gráfico, la biblioteca Swing ofrece un conjunto integral de componentes de interfaz. La ventana principal se construye mediante un contenedor de primer nivel `JFrame`; la entrada de texto plano se gestiona con `JTextField`; el campo especializado `JPasswordField` garantiza el tratamiento confidencial de credenciales; el botón `JButton` desencadena la acción del usuario; y el componente `JLabel` retroalimenta visualmente el estado del acceso. La documentación de referencia de Oracle enfatiza la vinculación de controles con receptores de eventos a través de la interfaz `ActionListener` [2].

La estimación de emisiones de gases de efecto invernadero depende del alcance metodológico y de los factores aplicables. Los inventarios internacionales categorizan las fuentes según emisiones directas e indirectas, donde el factor eléctrico responde a la mezcla de generación regional y temporal [3]. A fin de conservar la reproducibilidad y verificabilidad del ejercicio, se definen dos factores didácticos estables: 0.40 kg CO_2e/kWh para suministro eléctrico y 2.31 kg CO_2e/L para combustible regular, valores derivados y redondeados a partir de inventarios técnicos oficiales de emisiones [4].

3. Desarrollo

3.1. Investigación y análisis de la consulta

El desarrollo de la práctica tomó como punto de partida el análisis formal de los requerimientos descritos en la especificación técnica (`../Capturas/query.txt`). Dicha formulación delimita la problemática a resolver: diseñar un contrato único `HuellaCarbono` capaz de desacoplar el cálculo numérico de emisiones entre tipos heterogéneos, y construir un módulo gráfico interactivo de autenticación con componentes estándar de Swing.

La especificación formal fija los parámetros canónicos de prueba:

- **Edificio:** Evaluado con un consumo eléctrico anual de 2400.00 kWh y un factor de conversión regional de 0.40 kg CO_2e/kWh, produciendo una emisión analítica de 960.00 kg CO_2e.
- **Automóvil:** Evaluado sobre un trayecto de 480.00 km, un consumo nominal de 7.00 L/100 km y un factor de combustión de gasolina de 2.31 kg CO_2e/L. El consumo neto corresponde a 33.60 L, generando una huella analítica de 77.616 kg CO_2e (redondeada reglamentariamente a 77.62 kg CO_2e).
- **Bicicleta:** Modelada sobre un recorrido de 12.00 km bajo el alcance delimitado de emisiones directas operativas nulas (0.00 kg CO_2e).
- **Caso frontera:** Incorporación voluntaria de un edificio con consumo nulo (0.00 kWh) para auditar la estabilidad aritmética de la solución ante condiciones de borde.

El diseño del sistema garantiza la cohesión y separación de responsabilidades, organizando las entidades bajo el paquete institucional `mx.unam.fi.die.poo.g7.p8`.

3.2. Interfaz gráfica de login

La clase `LoginGUI` extiende de `JFrame` e implementa de manera directa la interfaz `ActionListener`. El formulario organiza un esquema matricial compuesto por un campo de texto para usuario, un campo protegido para contraseña (`JPasswordField`), un botón de ingreso (`JButton`) y una etiqueta de estado (`JLabel`). La arquitectura estática de la clase se presenta en el diagrama de clases de la Figura 3.1, mientras que la dinámica de interacción y captura de eventos se ilustra en el diagrama de flujo de la Figura 3.2. La validación compara las entradas frente a las credenciales didácticas (`usuario` y clave `poo8`), notificando el veredicto en la interfaz.

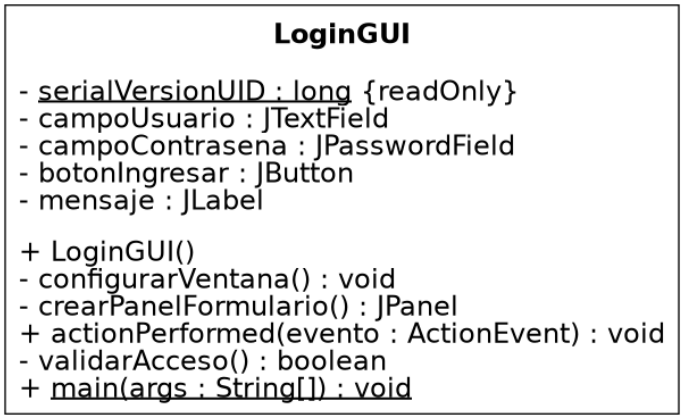


Figura 3.1: Diagrama UML de la clase LoginGUI.

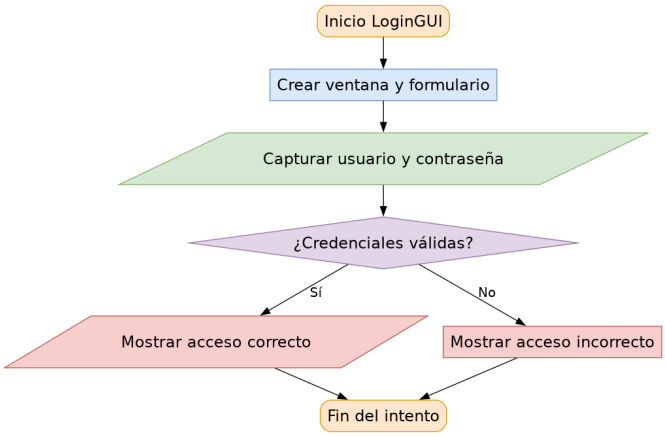


Figura 3.2: Flujo de captura, eventos y validación del inicio de sesión.

3.3. Contrato de huella de carbono

La interfaz `HuellaCarbono` estipula un contrato consistente en un único método: `double getHuellaCarbono()`. Al mantener la interfaz concisa y altamente cohesiva, los componentes clientes interactúan exclusivamente a través de la abstracción sin acoplarse a las estructuras internas de cada implementación. Como se aprecia en la Figura 3.3, el compartimento de atributos permanece vacío debido a que la interfaz no declara estado mutable. Por su parte, la Figura 3.4 modela el flujo operacional por el cual cada tipo concreto asume la obligación de resolver el cálculo.

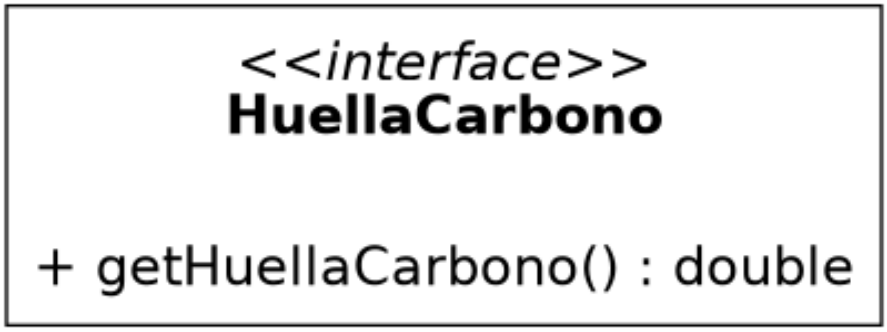


Figura 3.3: Diagrama UML de la interfaz HuellaCarbono.

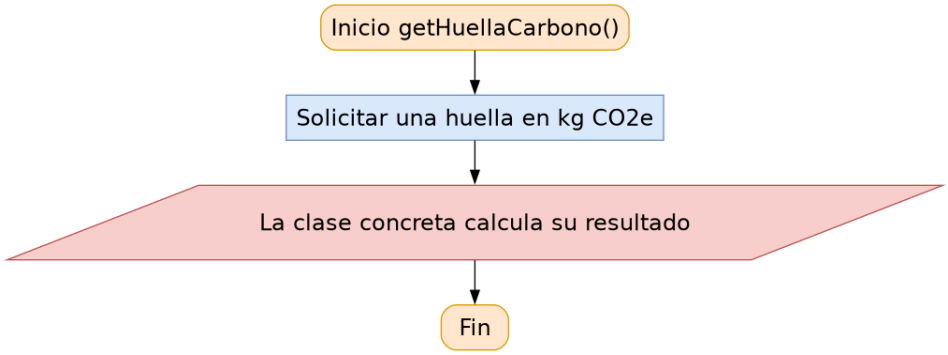


Figura 3.4: Flujo conceptual de delegación de la interfaz HuellaCarbono.

3.4. Clase Edificio

La clase `Edificio` cuantifica el impacto ambiental a partir del consumo anual de energía eléctrica y su factor de emisión correspondiente. La relación de cálculo adoptada es directa: $H = E \times F_e$, donde E denota los kilowatt-hora acumulados y F_e el factor de emisión eléctrica. Los datos miembros se mantienen bajo protección privada (`nombre`, `consumoAnualKwh` y `factorElectricidad`), proveyendo métodos accesores explícitos conforme al diagrama UML de la Figura 3.5. El algoritmo de evaluación de la huella se detalla en el diagrama de flujo de la Figura 3.6.

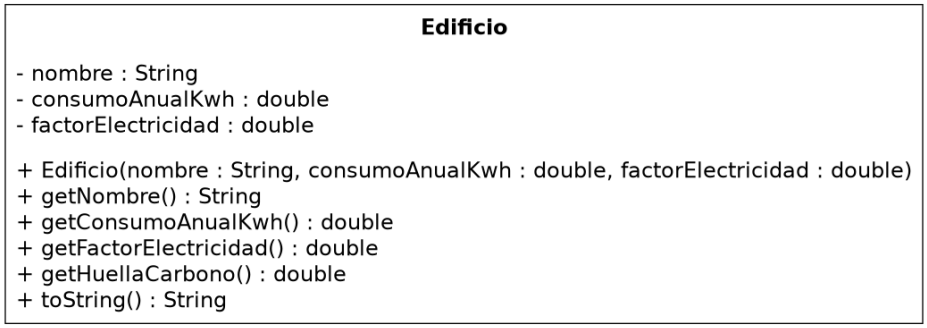


Figura 3.5: Diagrama UML de la clase Edificio.

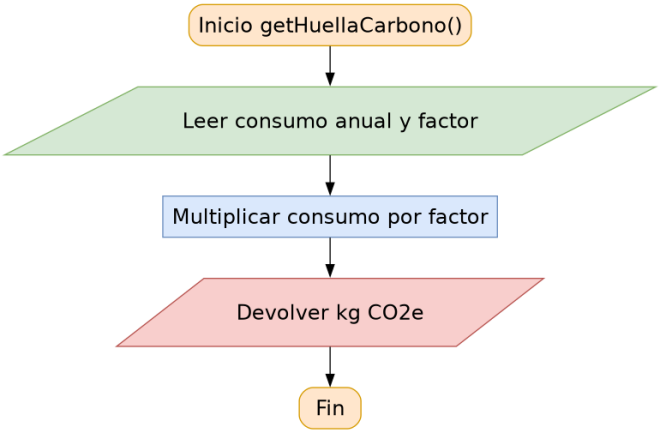


Figura 3.6: Flujo de cálculo de huella eléctrica en la clase Edificio.

3.5. Clase Auto

La clase `Auto` representa un vehículo terrestre automotor. El volumen de combustible quemado se evalúa preliminarmente mediante $L = (K \times R)/100$, donde K denota la distancia en kilómetros y R el rendimiento expresado en litros por cada cien kilómetros. Con dicho valor, la huella total se obtiene con la relación $H = L \times F_g$, empleando el factor de combustión de gasolina F_g. En la Figura 3.7 se observa el encapsulamiento de sus atributos y en la Figura 3.8 se diagrama la secuencia aritmética de dos fases que favorece la trazabilidad del cálculo.

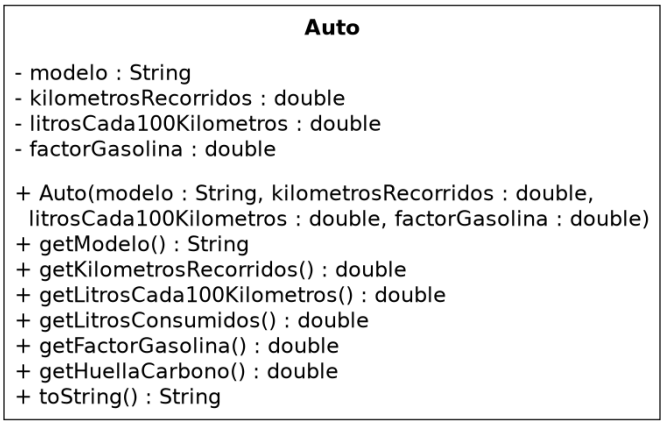


Figura 3.7: Diagrama UML de la clase Auto.

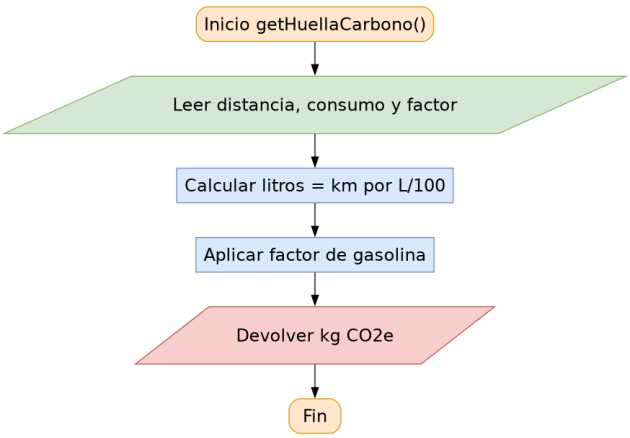


Figura 3.8: Flujo de cálculo de combustible y huella de la clase Auto.

3.6. Clase Bicicleta

La clase `Bicicleta` gestiona el tipo de unidad y el recorrido acumulado. En el modelo educativo formulado, el alcance se restringe exclusivamente a las emisiones operativas directas generadas durante el trayecto, las cuales resultan nulas al no existir combustión interna ($H = K \times 0.00 = 0$). La estructura de atributos encapsulados se exhibe en la Figura 3.9, y el diagrama de flujo correspondiente a su evaluación directa se ilustra en la Figura 3.10.

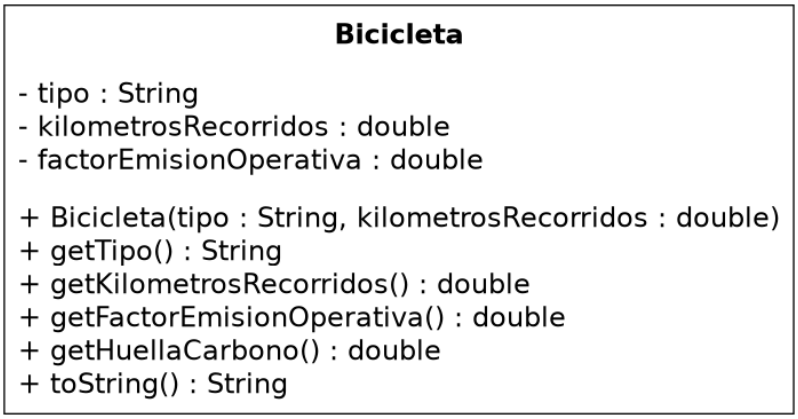


Figura 3.9: Diagrama UML de la clase Bicicleta.

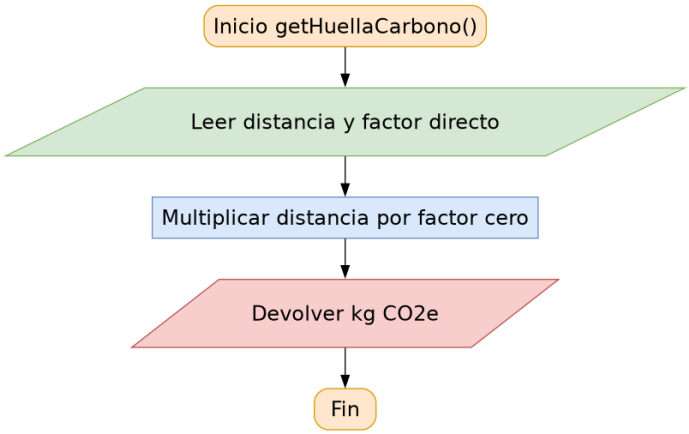


Figura 3.10: Flujo de evaluación de emisiones operativas de Bicicleta.

3.7. Clase principal y despacho polimórfico

La clase `Main` materializa el escenario de prueba instanciando los tres elementos solicitados, además de un caso de prueba frontera consistente en un edificio con consumo eléctrico nulo. Las cuatro instancias se agregan a una lista genérica de tipo `ArrayList<HuellaCarbono>`. Al recorrer la colección, el programa interactúa únicamente con el tipo general de la interfaz: invoca `getHuellaCarbono()` y la máquina virtual resuelve internamente el método especializado en tiempo de ejecución. La Figura 3.11 presenta la composición estática de la clase principal y la Figura 3.12 esquematiza el ciclo de recorrido polimórfico.

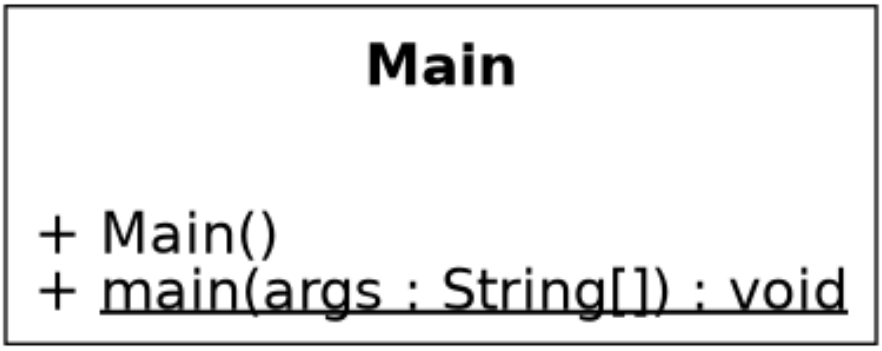


Figura 3.11: Diagrama UML de la clase principal Main.

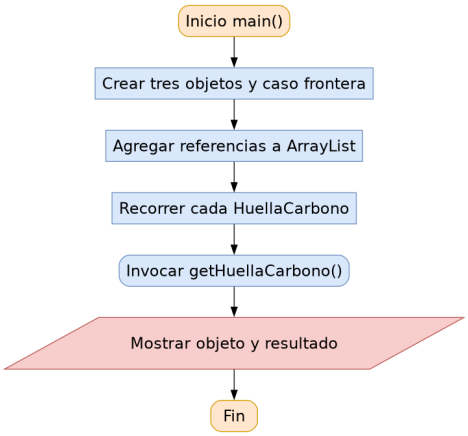


Figura 3.12: Flujo de instanciación, colección y recorrido polimórfico.

4. Resultados

4.1. Verificación cuantitativa de emisiones

Las pruebas experimentales se contrastaron frente a los cálculos analíticos teóricos previamente formulados. El edificio con consumo anual de 2400.00 kWh y factor 0.40 kg CO_2e/kWh reporta exactamente 960.00 kg CO_2e. El automóvil que recorrió 480.00 km a 7.00 L/100 km consumió 33.60 L de gasolina; multiplicado por el factor 2.31 kg CO_2e/L arrojó 77.616 kg CO_2e, registrando 77.62 tras redondeo canónico a dos posiciones decimales. La bicicleta produce 0.00 kg CO_2e por definición operativa directa, y el caso frontera con consumo cero arroja 0.00 kg CO_2e, ratificando que consumos nulos no introducen valores espurios. Los resultados cuantitativos se compendian en el Cuadro 1.

Tabla 1: Comparación cuantitativa entre valores esperados y observados en ejecución.

Instancia	**Parámetros de entrada**	**Esperado**	**Observado**	**Dictamen**
Edificio Escuela	2400.00 kWh, factor 0.40	960.00	960.00	Exitoso
Auto Sedán	480.00 km, 7.0 L/100 km, factor 2.31	77.62	77.62	Exitoso
Bicicleta urbana	12.00 km, factor operativo 0.00	0.00	0.00	Exitoso
Edificio frontera	0.00 kWh, factor 0.40	0.00	0.00	Exitoso

La ejecución comprobó la correcta emisión textual de cada instancia y la resolución unificada de huella a través de la interfaz. La evidencia gráfica de la compilación limpia con el compilador `javac` se documenta en la Figura 4.1, mientras que la verificación de empaquetado del archivo ejecutable `Practica8.jar` se muestra en la Figura 4.2.



Figura 4.1: Compilación limpia de los seis archivos fuente Java hacia la carpeta test.

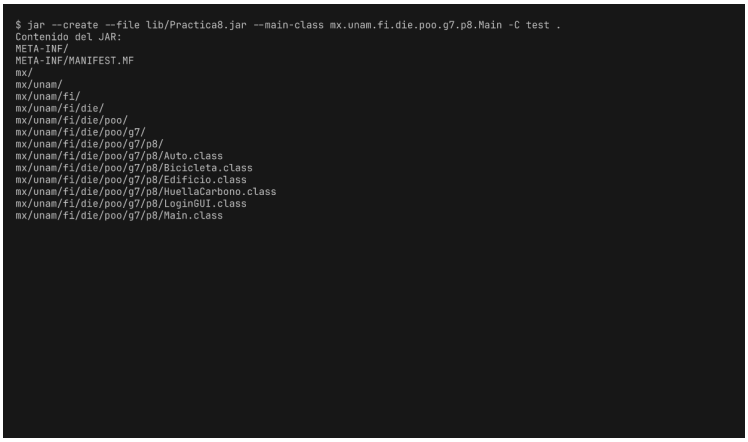


Figura 4.2: Generación y verificación de contenido del archivo ejecutable Practica8.jar.

4.2. Ejecución desde classpath y archivo JAR

La ejecución del programa se validó bajo dos modalidades idénticas: directamente desde el árbol de clases compiladas en `test/` (Figura 4.3) y a través del archivo empaquetado `lib/Practica8.jar` provisto de manifiesto de ejecución (Figura 4.4). Ambas vías arrojaron exactamente las cuatro lecturas evaluadas: 960.00, 77.62, 0.00 y 0.00 kg CO_2e, confirmando la paridad operativa entre ambos esquemas.

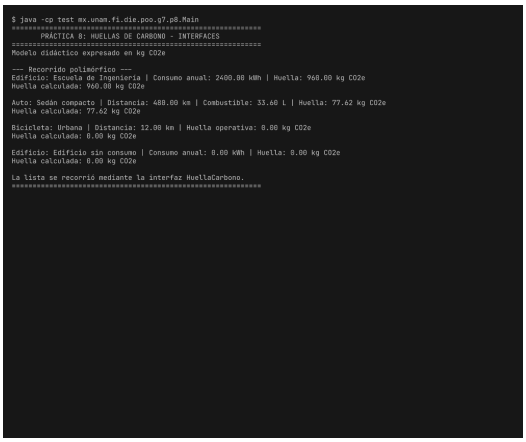


Figura 4.3: Salida de consola observada al ejecutar Main desde la carpeta de clases compiladas.

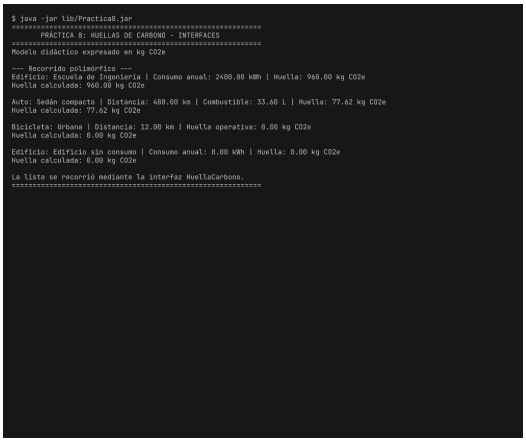


Figura 4.4: Salida de consola observada al ejecutar la aplicación desde Practica8.jar.

4.3. Documentación técnica y estructura de entrega

Se comprobó la generación independiente de documentación Javadoc para cada una de las seis entidades (`HuellaCarbono`, `Edificio`, `Auto`, `Bicicleta`, `Main` y `LoginGUI`) dentro de subdirectorios aislados en `doc/Javadoc`. La carpeta `test/` aloja exclusivamente los archivos binarios `.class` generados durante la compilación, y `lib/` conserva el archivo JAR autónomo de la práctica.

5. Conclusiones

La implementación de la interfaz `HuellaCarbono` confirmó la eficacia del polimorfismo por interfaz en Java: tres clases con modelos de dominio divergentes se articularon armónicamente bajo un contrato unificado sin requerir jerarquías de herencia forzadas. Cada entidad resguardó sus atributos mediante encapsulamiento estricto y aplicó su propia fórmula de emisiones, permitiendo que la colección cliente opere de manera desacoplada y genérica.

Los resultados experimentales observados en ejecución coincidieron plenamente con los valores analíticos formulados antes de la prueba. El caso de frontera ratificó la estabilidad aritmética de la implementación ante entradas nulas, mientras que las pruebas cruzadas entre classpath y archivo JAR garantizaron la reproducibilidad del entorno sin inconsistencias de configuración.

La clase `LoginGUI` completó los requerimientos de interfaz gráfica interactiva demostrando el uso de campos protegidos `JPasswordField` y la delegación de eventos mediante `ActionListener`. En conjunto, la solución articula con precisión los principios fundamentales de abstracción, polimorfismo, verificación numérica y presentación formal de software de acuerdo con los criterios académicos de la asignatura.

Bibliografía

[1] Oracle Corporation, *The Java Language Specification, Java SE 25 Edition: Interfaces*. Austin, TX, EE. UU.: Oracle Corp., 2025.

[2] Oracle Corporation, "Creating a GUI With Swing", en *The Java Tutorials*. Redwood City, CA, EE. UU.: Oracle Corp., 2024. [En línea]. Disponible en: `https://docs.oracle.com/javase/tutorial/uiswing/`. [Accedido: 29-oct-2026].

[3] Agencia de Protección Ambiental de los Estados Unidos (EPA), *Emissions & Generation Resource Integrated Database (eGRID)*. Washington, D.C., EE. UU.: EPA, 2024.

[4] Agencia de Protección Ambiental de los Estados Unidos (EPA), *Emission Factors for Greenhouse Gas Inventories*. Washington, D.C., EE. UU.: EPA, 2024.